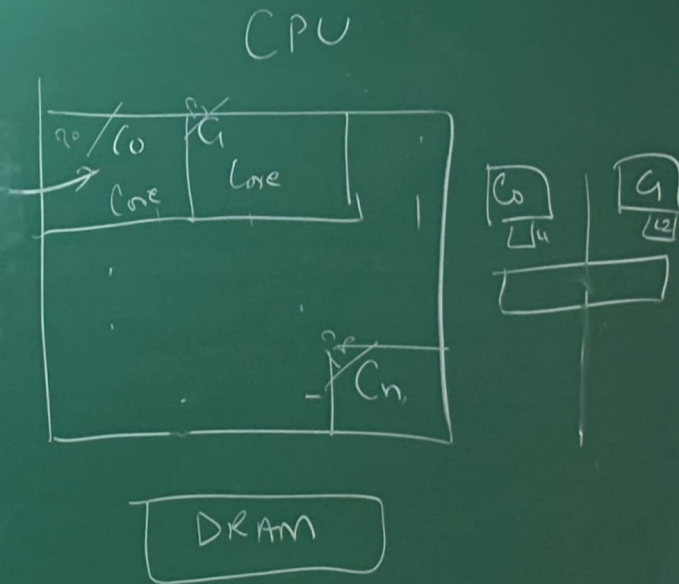
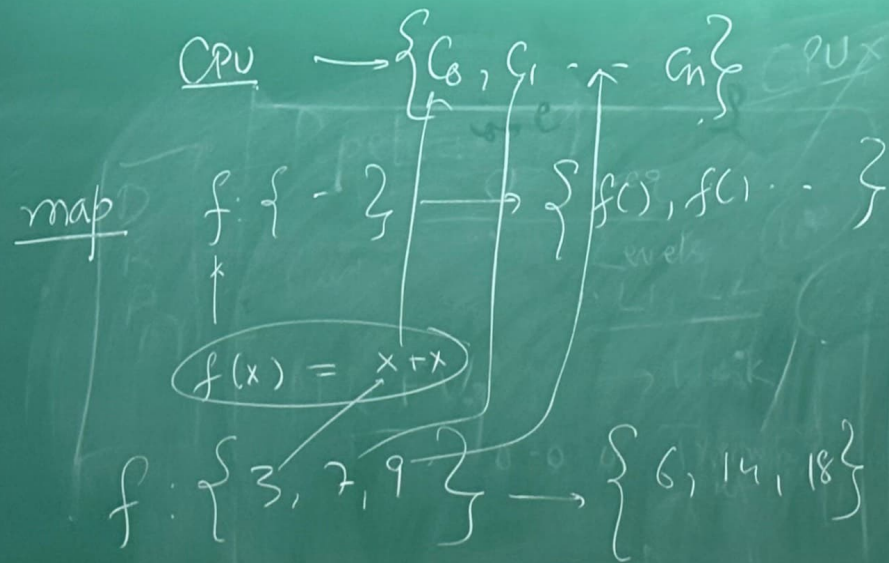
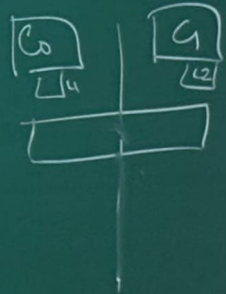
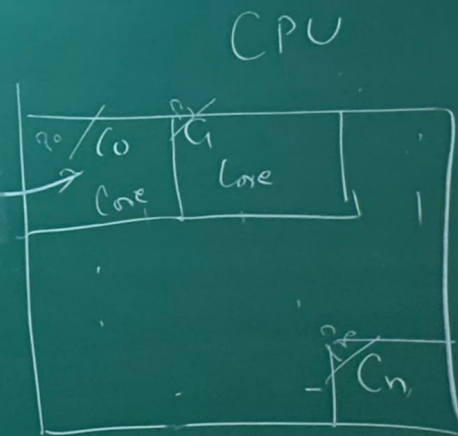
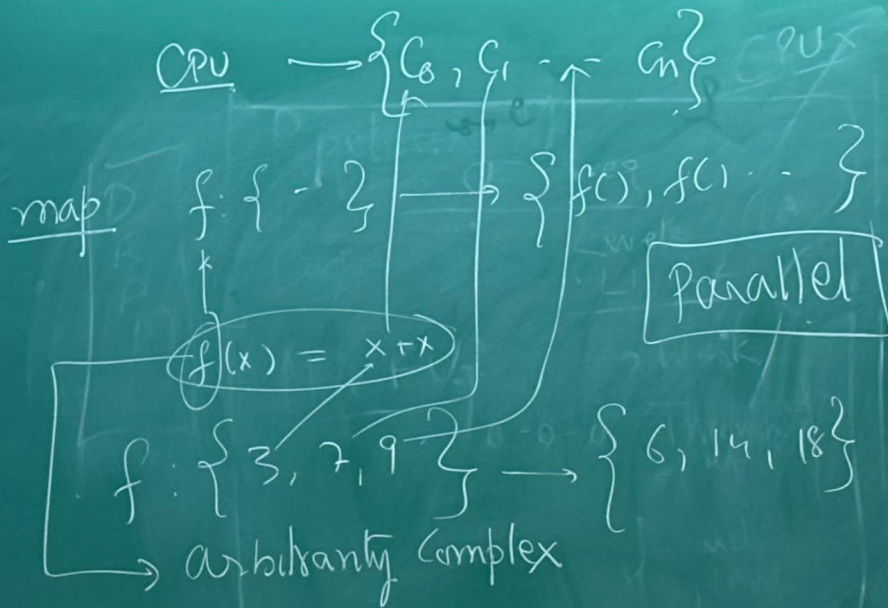






DRAM

CPU $\{C_0, C_1, \dots, C_n\}$ $\rightarrow$ PUs

map

emb arr as singly ||

$\{10^6\}$

Serialized
(of input)

Sum of a Large Array $\rightarrow O(n)$

Partial Sum

Reduction

Speed Up

Procs

C_0

CPU
P

Procs

C_P



$\{p_i\}$

p_i

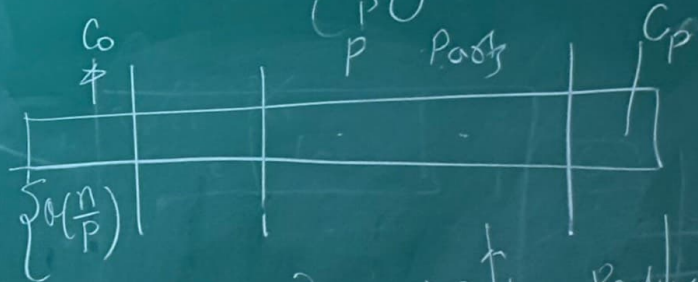
DRAM

$$T_{par} = T_{seq} f + (1-f) \frac{T_{seq}}{P}$$

$$T_{par} = \frac{T_{seq}}{P}$$

$$\frac{T_{seq}}{T_{par}} = \text{Speed Up}$$

→ Proc
CPU
P



n) ParahSum

Reduction

$\{ p_i \}$
↓
P

map - Reduce
||
Serial

Multiprocessor Types

- Loosely coupled multiprocessors
 - No shared global memory address space
 - Multicomputer network
 - Network-based multiprocessors
 - Usually programmed via message passing
 - Explicit calls (send, receive) for communication
- Tightly coupled multiprocessors
 - Shared global memory address space
 - Traditional multiprocessing: symmetric multiprocessing (SMP)
 - Existing multi-core processors, multithreaded processors
 - Programming model similar to uniprocessors (i.e., multitasking uniprocessor) except
 - Operations on shared data require synchronization

Main Design Issues in Tightly-Coupled MP

- Shared memory synchronization
 - How to handle locks, atomic operations
- Cache coherence
 - How to ensure correct operation in the presence of private caches keeping the same memory address cached
- Memory consistency: Ordering of all memory operations
 - What should the programmer expect the hardware to provide?
- Shared resource management
- Communication: Interconnects

Main Programming Issues in Tightly-Coupled MP

- Load imbalance
 - How to partition a single task into multiple tasks
- Synchronization
 - How to synchronize (efficiently) between tasks
 - How to communicate between tasks
 - Locks, barriers, pipeline stages, condition variables, semaphores, atomic operations, ...
- Contention
- Maximizing parallelism
- Ensuring correct operation while optimizing for performance

25 Sept

25 Sept
Till now we
did,

increases

CPU

connected to DRAM somehow

Lots of Transistors

Pipeline Freq.

Cache Levels
L₁ L₂

More F-V.

work/ ↑
Throughput

10-0-0
(out of order)

All of this has certain limit (old)

All of this has certain limit (old) m

More FU try to take advantage of your prog. but increases hazardously Dependency

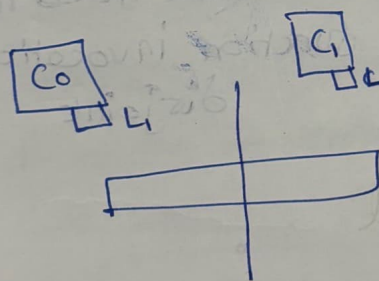
So what to do with extra transistors

So wait to see
why don't we put in copies of this?
correct of cores

Introducing concept of cones

Introducing concept of
Each core may/may not have separate L2 cache
(Depends on workload)

How much of shearing
you want to
allow?



These cores should be able to do more work

These processors $C_1, C_2, \dots, C_n$ connected over a network

$CPU \rightarrow \{c_0, c_1, \dots, c_n\}$

Application $\rightarrow$ map $f: \{ \} \rightarrow \{ f(), f(), \dots \}$

$$f(x) = x + x$$
$$f \{3, 7, 9\} = \{6, 14, 18\}$$

→ we want to be able to put these cores in parallel

$$f(x) = x + x$$

can be Arbitrarily Complex

[Might be entire OS by itself]

can be a program / has to be an algorithm

OS never terminates

(prog)

Algo - has to terminate

for each invocation data is disjoint

ex $\{3, 5, 7, 9\}$
 $\uparrow \quad \uparrow \quad \uparrow \quad \uparrow$
 $P_1 \quad P_2 \quad P_3 \quad P_4$

It need not be Avg, it could be working on Avg.

* Data need not be disjoint
function invocation must be disjoint

map
↳ embarrassingly parallel

$\{10^6\}$

All 10^6 elements must be available for each processor given you have 10^6 processors

$\{10^6\} \rightarrow$ serialization (of input)

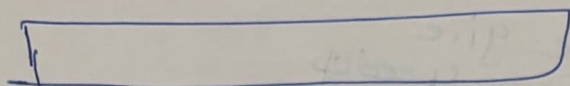
why in parallel $\rightarrow$ I am looking for speedup

Ideal Speedup = No. of processors

$$\text{Speedup} < p$$

map is what we can ^{do} but applications can not do map

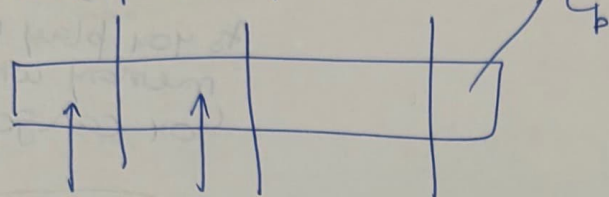
Sum of large Array = $O(n)$



Large Array



Break into p parts



inside this algo takes $O(n/p)$ time and each of these sums happening in parallel

$\Rightarrow$ have p partial sums.



have to do Reduction {Rate limiting step}

and

we cannot do reduction in parallel

Even if input takes no time, final result require Reduction thus mapreduce framework is embarrassingly parallel but Reduction is embarrassingly serial

and therefore idle speedup is limited by Reduction

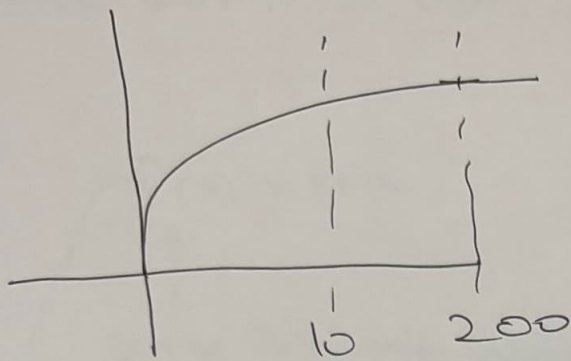
$$\text{speedup} \leq p$$

$$\text{Speedup} = \frac{T_{\text{seq.}}}{T_{\text{parallel}}} \leq p$$

So every program will have a serial & a parallel portion

$$T_{\text{par}} = T_{\text{seq.}} \cdot f + (1-f) \frac{T_{\text{seq.}}}{p} \Rightarrow \frac{1}{S} = \frac{T_{\text{par}}}{T_{\text{seq.}}} = f + \frac{(1-f)}{p}$$

$$\Rightarrow f + \frac{(1-f)}{p}$$



Curve give
Some speedup
then
it reduces

(Book has problem
on this)

As you play with
memory what speedup
you can get

Most of CS is about Sorting / Searching / Numerical Computation
Reduction

Give Example of Application where all 3 used?

→ online shopping application?
(Building) - (Software part)

Coins & stamps

T-shirts

Search (query) → Sort (size) → Reduction (cart)

Parallel Sort



at some point
would get limited
by fraction which
is sequential



α_{seq}

$$1 - \alpha_{seq} = \alpha_p$$

{ Searching } parallel
Sort
{ Reduction }

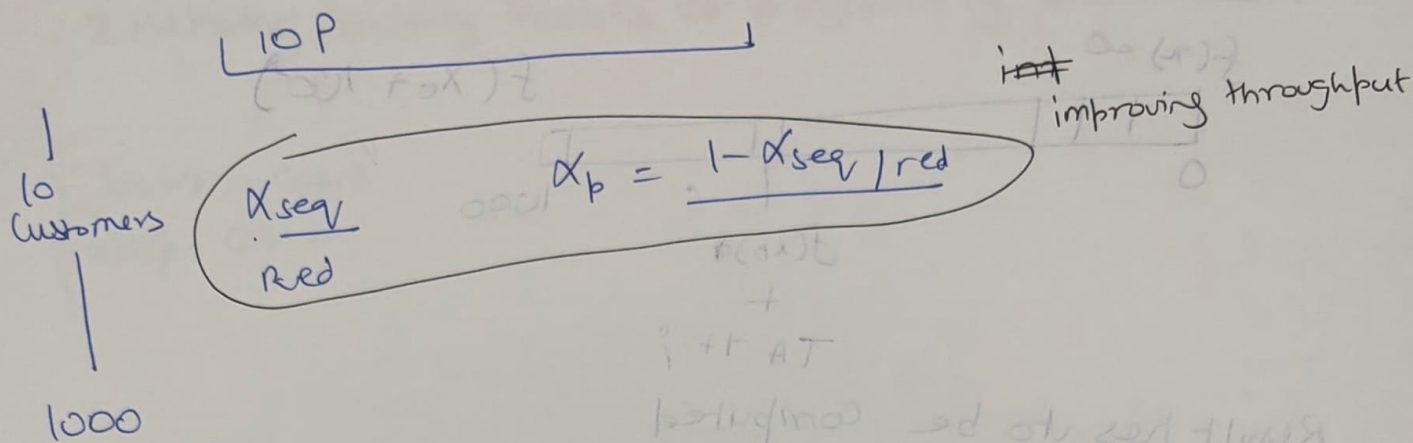
Let's say

10 processors

10 - Customer at start

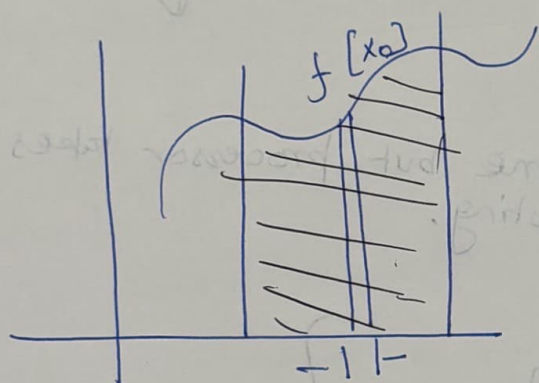
$$\frac{(t-1)}{p} + \frac{1}{p} = \frac{t}{p} = \frac{1}{p} \leq \frac{t}{p} + \frac{1}{p} = \frac{t+1}{p}$$

$$\frac{(t-1)}{p} + 1 \leq \frac{t}{p} + 1$$



work/throughput we want to accomplish in same amount of time
 - throw more res to get ~~more~~ more work done

$$\alpha_p = 1 - \alpha_{seq} / red$$



What is the $h \rightarrow 0$
 $n = \frac{b-a}{h}$

$n_1 = 10$ processors

$n_2 = 1000$ parts

more processor = more work done faster

Time - same, we will do more work

$a - a + h, a + 2h, \dots$

Input which it operate on given by press = id

[time may slightly increase]

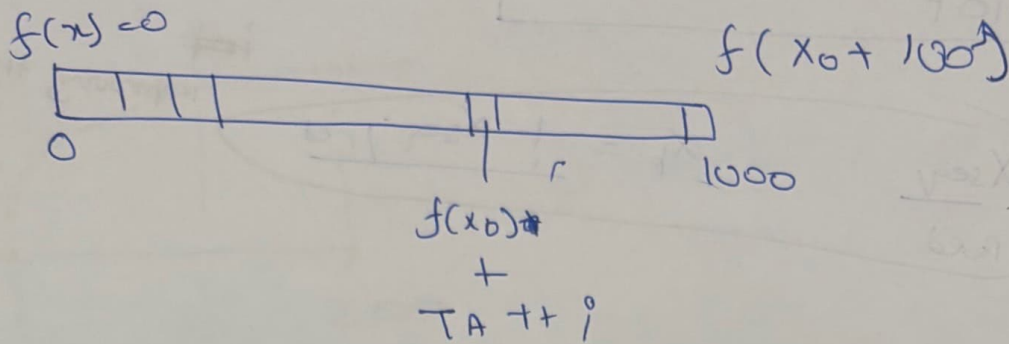
[Filter mesh better results]

Gustafson Barrier Law

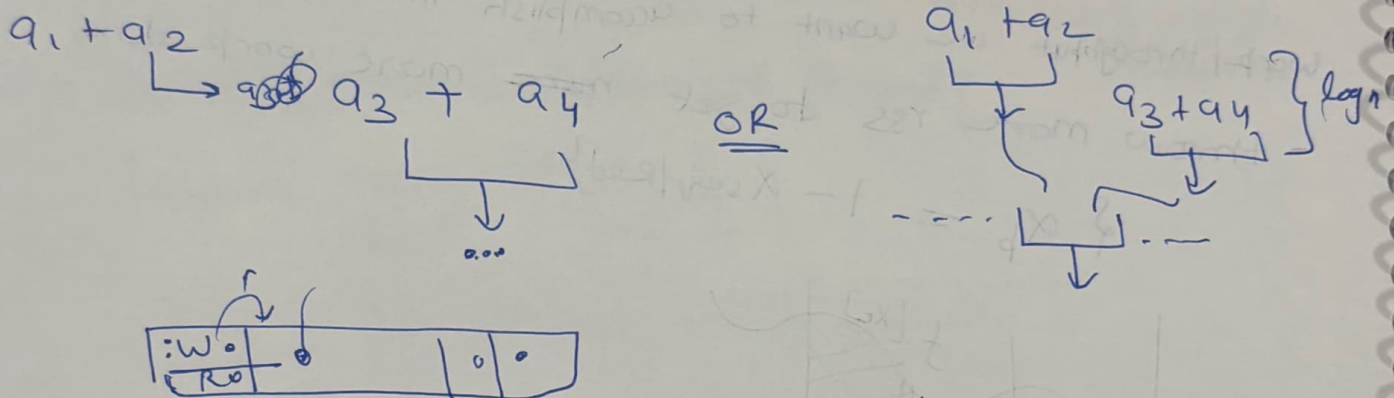
$n_1 \rightarrow n_2$ You get Advantage if there is Degree of parallelism.

$$\nabla^2 f = ? - 3D$$

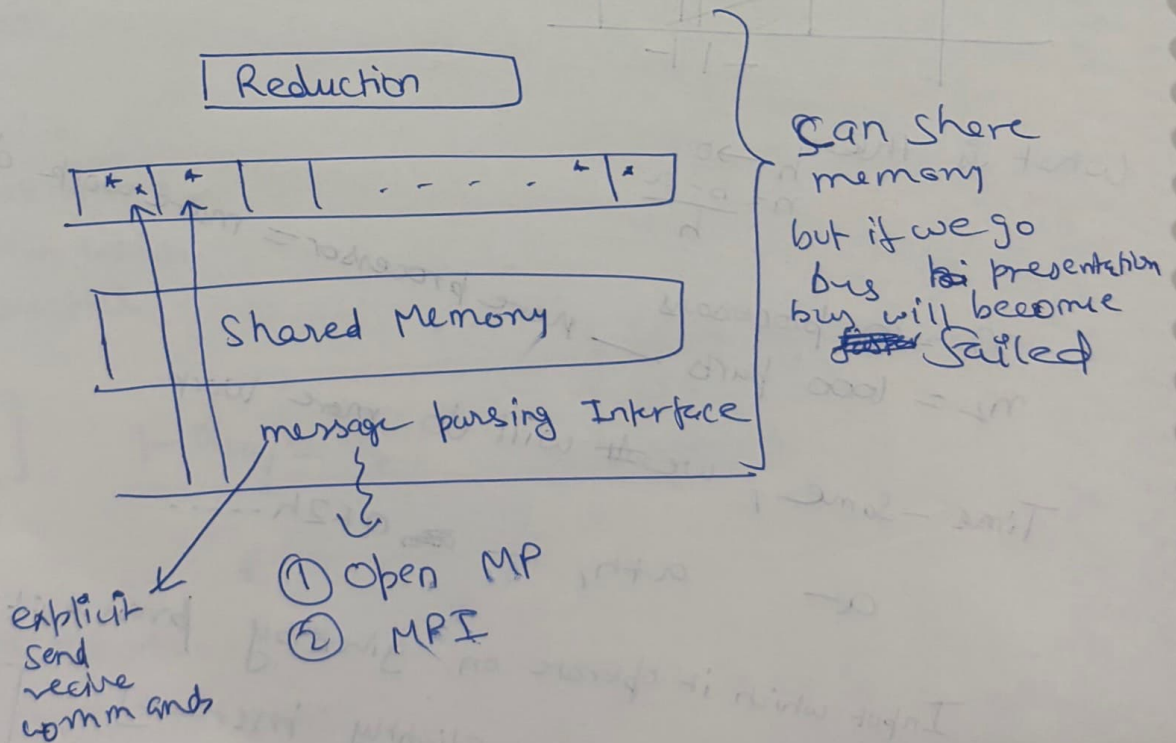
Reduction



Result has to be computed
 $\hookrightarrow$ 1000 values



~~Result~~ Results computed same time but processor takes from shared memory more computing.



Compiler figure out which can relate to processor